

GLITZY

DOC NO. RL-360-A REV. A RIPOSTE LABORATORIES

CONTENTS

▶ Why they are one repository	2
▶ Layout	3
▶ Running Etsch	3
▶ Running the studio	4
▶ Deploying	4
▶ Licence and attribution	5

Make glitch art out of chains of operations, then cut it out with a vinyl cutter.

Two tools, one pipeline:

Glitzy studio

source.flow
→ codec.smear
→ colour.palette
→ matte.border

build the artwork
—————→
hand off a sheet

Etsch

place on the sheet
set bleed and margins
trace the cut contour
registration marks
↓
print file + cut file

Glitzzy is the studio. A project is a stack of chains of typed operations, evaluated on a server and cached at every node, so editing the last node of an eight-node chain recomputes exactly one node. Seven of its operations corrupt the MPEG-2 bitstream itself rather than the pixels – real datamoshing, via `ffglitch`.

Etsch is the cutting step. It takes finished artwork, lays it out on a print sheet, and generates the contour paths a cutter follows. It runs entirely in the browser: no image it opens is ever uploaded anywhere.

Etsch is useful on its own – most people who want to cut stickers do not want to datamosh anything first – so it deploys on its own and stands alone. The studio is what turns it into a pipeline.

Hosting: Etsch is ready to deploy to Cloudflare as static assets and is **not currently published** – `wrangler.toml` is configured, so one `npx wrangler deploy` publishes it whenever you want. The studio is self-hosted and stays on a private network by design; see `studio/README.md` RL-360-STUDIO.

SEC.01

WHY THEY ARE ONE REPOSITORY

They were two projects, and the seam between them kept costing time.

The studio hands a sheet to Etsch as an `etsch/1` document. That format is a contract between the two, and while they lived in separate repositories nothing checked that the writer and the reader agreed – each side asserted against its own copy of the same string. Worse, the handoff route itself was built, deployed, and then committed to neither repository for a day, because it touched both sides and belonged to no one.

One repository, one test run, one place the format is defined.

SEC.02

LAYOUT

etsch/	the cutting tool – browser only, no runtime dependencies
public/	the app as it deploys: ES modules, HTML, CSS, no build step
tests/	96 checks: logic, DOM, real-canvas rendering, the handoff
studio/	the chain studio
glitzyd/	the Python engine: graph, cache, ops, HTTP API
rack/	ffedit qjs scripts, one per codec operation
web/	the studio UI – native ES modules, no build step
tests/	backend checks against real ffglitch, UI checks in Chromium
deploy/	the reverse-proxy site file and deploy scripts
wrangler.toml	deploys etsch/public to Cloudflare as 'glitzy'
package.json	the Node half; the studio is Python and needs no npm

studio/ mirrors the engine host's own layout, so a path that works in the repo works on the box.

SEC.03

RUNNING ETSCH

Needs Node 22 or newer. The app is plain ES modules with no build step; Node is only for the dev server, the tests and deployment.

SH

npm ci	
npm run dev	# http://localhost:5173
npm test	# 96 checks

SEC.04

RUNNING THE STUDIO

The studio needs Python 3.11+, NumPy, Pillow, and ffglitch. It is a server because the cache is the point – see `studio/README.md`^{RL-360-STUDIO} for setup, the operation reference, and a long list of things that will bite you.

SEC.05

DEPLOYING

`npx wrangler deploy` publishes `etsch/public` to Cloudflare Workers as static assets and attaches `glitzy.ripostelabs.xyz`. **Nothing is published yet** – the config is ready and the command has not been run. See `docs/pdf/deploy.pdf`.

There is no Worker code: with `assets` and no `main`, nothing runs per request. Etsch makes no backend calls of any kind – the only `fetch()` in it is a relative, same-origin `/handoff/<id>`, a route that exists on the internal deployment and not on Cloudflare, where it 404s and is handled. So publishing Etsch exposes no server and reaches nothing private.

The studio is the opposite and is treated as such. It accepts uploads and spawns subprocesses on them, so it stays on a private network behind an authenticating reverse proxy. Nothing in this repository puts it on the open internet, and no part of the public deployment talks to it.

SEC.06

LICENCE AND ATTRIBUTION

Glitzzy is MIT-licensed – see [LICENSE](#).

Almost none of the interesting work here is ours. The bitstream corruption is `ffglitch`, the array work is `NumPy`, the imaging is `Pillow`, and the studio sets its type in `JetBrains Mono`. [docs/pdf/third-party.pdf](#) lists every one of them with its licence, and is specific about the two that ask something of you:

- ▶ **`ffglitch` is GPL-2.0-or-later.** Glitzzy runs it as a separate process and does not ship it, so Glitzzy's own MIT licence stands – but if **you** build a distribution that bundles the `ffglitch` binaries, the GPL attaches to that.
- ▶ **JetBrains Mono is under the SIL Open Font License 1.1**, and is the one dependency this repository actually redistributes, so its licence ships next to it at [studio/web/fonts/JetBrainsMono-OFL.txt](#). If the font moves, that file moves with it.

Cricut, Silhouette and Roland are trademarks of their owners and appear only to name which machine a profile is for. This project is not affiliated with any of them.

SOURCE README.md SHA 16a5cf703407 BUILT 2026-09-07